Reviewer Pack — Plethysm Coefficient Gap in Symmetric Powers of Permanent vs…

Entry 3424afae2a44 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 21:14:35 UTC

Plethysm Coefficient Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: `3424afae2a44` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 21:14:35 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Monotone Circuit Complexity

Statement:

> For all n ≥ 2, the plethysm coefficient of Sym^k(perm_n) exceeds that of Sym^k(det_m) by a factor of Ω(2^{n/2}) for m < n^{1.5} and k = ⌊n/2⌋.

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor powers into irreducible representations; plethysm coefficients measure how these decompose under symmetric power operations. Permanent and determinant have distinct orbit closures under GL(n), suggesting their symmetric power decompositions should exhibit exponential separation in specific coefficients.

Taxonomy category: `AVG_TO_WORST_CASE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3c186a2ff3211838`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def young_tableaux(n):
        if n == 1:
            return [[]]
        tableaux = []
        for i in range(1, n):
            for t in young_tableaux(n - i):
                tableaux.append([i] + t)
        return tableaux

    def hook_length_formula(shape):
        n = len(shape)
        fact = math.factorial
        numerator = fact(n * (n + 1) // 2)
        denominator = 1
        for row in shape:
            for i, cell in enumerate(row):
                denominator *= (cell + i + 1)
        return numerator / denominator

    def plethysm_coefficient(shape, k):
        n = len(shape)
        if k == 0:
            return 1
        coeff = 0
        for t in young_tableaux(n):
            if len(t) >= k:
                coeff += hook_length_formula([t[i] + 1 for i in range(k)])
        return coeff / (k * hook_length_formula(shape))

    n = random.randint(2, 40)
    m = int(n ** 1.5)
    k = n // 2

    perm_coeff = plethysm_coefficient([n - k + 1] + [1] * (n - k), k)
```

```
    det_coeff = plethysm_coefficient([n - k + 1] + [1] * (n - k), k)

    ratio = perm_coeff / det_coeff

    conjecture_holds = ratio > 2 ** (n / 2)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} < 2^{n/2}"

    return {
        "metric_name": "plethysm_coefficient_ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f38910c.py", line 76, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f38910c.py", line 54, in run_trial
    perm_coeff = plethysm_coefficient([n - k + 1] + [1] * (n - k), k)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f38910c.py", line 47, in plethysm_coef
    coeff += hook_length_formula([t[i] + 1 for i in range(k)])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f38910c.py", line 36, in hook_length_f
```

```
    for i, cell in enumerate(row):
                   ^^^^^^^^^^^^^^
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with TypeError before producing results; cannot evaluate support fraction or counterexamples. | next: Fix the hook_length_formula parameter type error in the test code to enable proper execution

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109825
2	propose	ollama_remote	qwen3:8b	0	0	33010
3	preregistration	ollama_remote	qwen3:8b	0	0	24185
4	novelty	ollama_remote	qwen3:8b	0	0	20673
5	novelty	ollama_remote	qwen3:8b	0	0	15694
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15150
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8368
8	judge	ollama_remote	qwen3:8b	0	0	20797

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 247700 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/3424afae2a44.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3424afae2a44.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3424afae2a44.tar.gz` (if generated)